

Can Small Language Models Serve Ordinary Users? A Lower-Bound Benchmark under Non-Expert Prompts and Edge Constraints

Anonymous submission

Abstract

Small language models are increasingly promoted as low-cost, private, and edge-deployable alternatives to cloud-scale LLMs. Yet most evaluations assume favorable conditions: expert-written prompts, clean benchmark inputs, and high-precision inference. This overlooks the lower-bound regime in which small models are used by non-expert users under quantized or compressed deployment. We introduce **PEEL (Prompt-Edge Evaluation Ladder)**, a lightweight all-in-one automated benchmarking framework for evaluating small language models under joint prompt-quality and edge-deployment constraints. PEEL automates model/config selection, prompt-tier construction, inference, offline scoring, and aggregation across a two-dimensional ladder: prompts range from expert to broken, while deployment ranges from reference GPU inference to low-bit CPU GGUF inference. We evaluate 16 small and efficient models across QA, truthfulness, hallucination discrimination, safety behavior, and multi-turn dialogue. Our results show that prompt degradation is often more damaging than quantization. After controlling for output-scoring artifacts, TruthfulQA accuracy changes by only about one point from full precision to 4-bit inference under expert prompts. In contrast, SQuAD v2 exact match falls from roughly 30% under expert prompts to 2–5% under broken prompts across all models and deployment settings. We also find that prompt robustness is strongly shaped by post-training quality: at matched 1.5B scale, the strongest instruction-tuned hybrid model scores 8.20 on MT-Bench, compared with 4.06 for a same-size Transformer. Finally, keyword-based safety diagnostics show high unsafe compliance, with attack success rates around 20–21% and only moderate validation agreement. PEEL exposes the gap between clean benchmark performance and ordinary-user edge reliability.

Introduction

Small language models (SLMs) are increasingly positioned as practical alternatives to cloud-scale large language models (LLMs). With fewer parameters and lower memory requirements, they can run on consumer laptops, mobile devices, and other resource-constrained hardware, reducing dependence on cloud APIs for latency-sensitive, low-cost, and privacy-sensitive applications. This makes SLMs attractive not only as smaller models, but also as a possible mechanism for broadening access to language technologies under realistic hardware, connectivity, and privacy constraints.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

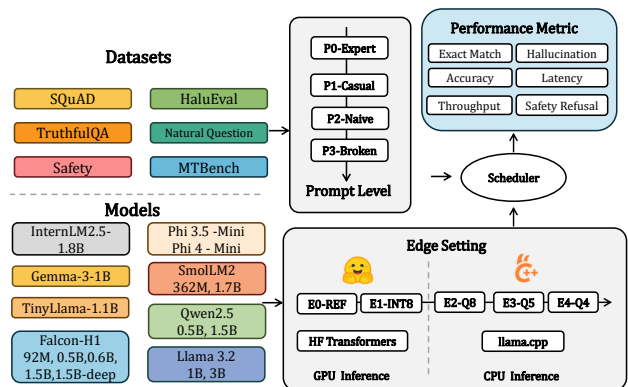


Figure 1: Overview of PEEL, which evaluates sub-4B models across prompt-quality tiers (P0–P3) and edge settings (E0–E4), measuring accuracy, hallucination, safety, quality, latency, and throughput.

Yet broad access changes what evaluation should measure. For ordinary users, a model’s practical value is not determined only by its best-case benchmark score under expert prompting and full-precision inference, but by what capability remains when interaction and deployment conditions are imperfect. Prior robustness work similarly argues that worst-case or perturbed prompt performance provides an important reliability signal, rather than a nuisance variation to be averaged away (Zhu et al. 2023; Cao et al. 2024). This perspective is especially important for SLMs: non-expert users may provide vague, colloquial, underspecified, or malformed prompts, while local deployment often relies on compressed or quantized inference because of limited memory, cost, energy, or connectivity. A model that appears capable under ideal prompts and server-class inference may therefore fail in the very regime where SLMs are meant to be useful.

We argue that SLM evaluation should report not only upper-bound benchmark performance, but also *lower-bound reliability*: how models behave when user input quality and deployment quality degrade together. Existing evaluations cover parts of this space, but rarely their interaction. General LLM benchmarks measure broad capability under standardized prompts and stable inference settings. Edge and on-device benchmarks move evaluation closer to resource-

constrained deployment, but typically hold prompting conditions fixed. Conversely, prompt-robustness studies perturb prompt quality while leaving the serving stack largely unchanged. As a result, we have limited evidence about the joint effect of non-expert prompting and edge deployment constraints—precisely the regime occupied by ordinary users running small models locally.

We close this gap with **PEEL (Prompt–Edge Evaluation Ladder)**, a lightweight all-in-one automated benchmarking framework for lower-bound assessment of small language models. PEEL automates the full benchmark loop: registry-based model and configuration selection, prompt-tier construction, inference, offline scoring, and summary aggregation into reproducible run records. Its core protocol crosses four prompt-quality tiers, from expert task instructions to broken ordinary-user prompts, with five edge-deployment settings, from full-precision GPU inference to aggressive low-bit local inference. Across this ladder, PEEL evaluates task accuracy, hallucination discrimination, safety behavior, multi-turn instruction-following quality, latency, and throughput. It asks a deployment-facing question: what remains reliable when both the user and the serving stack are imperfect?

We evaluate 16 sub-4B models across eight model families, covering extractive reading comprehension, open-domain question answering, truthfulness, hallucination discrimination, safety behavior, and multi-turn instruction following. Our central finding is that small models do not simply become weaker under edge constraints; they become more conditional. Reliability depends sharply on task format, prompt quality, runtime-interface fidelity, and the scoring assumptions used to interpret outputs. Across models and deployment settings, prompt degradation is often more damaging than bit-width reduction. On SQuAD v2, exact match falls from roughly 30% under expert prompts to **2–5%** under broken prompts across every edge setting, indicating a near-total prompt-driven collapse in span-based reading comprehension. In contrast, after controlling for output-scoring artifacts, TruthfulQA-MC1 changes by only about one point from full precision to 4-bit inference under expert prompts. These results suggest that, for many practical SLM deployments, the main reliability bottleneck is not quantization alone, but the interaction between task format, user prompt quality, and runtime assumptions.

Contributions. This paper makes the following contributions:

- **Automated PEEL framework.** We introduce a lightweight all-in-one benchmarking framework that automates model/config selection, prompt-tier construction, inference, offline scoring, and aggregation for lower-bound evaluation of small and edge LLMs.
- **Prompt–Edge Evaluation Ladder.** We define a two-axis protocol that crosses four prompt-quality tiers with five edge-deployment settings, and use it to evaluate 16 sub-4B models across QA, truthfulness, hallucination, safety, and dialogue tasks.
- **Prompt–edge reliability analysis.** We show that prompt degradation can dominate bit-width reduction for format-

sensitive tasks, while multiple-choice truthfulness is relatively stable once scoring artifacts are handled.

- **Reproducible artifacts.** We release PEEL prompts, deployment configurations, scoring scripts, per-run outputs, and result grids, including machine-readable summaries for future lower-bound evaluation of edge-deployed SLMs.

Related Work

Small language models, edge deployment, and quantization. Large language models have achieved strong performance across reasoning, question answering, and instruction-following tasks, but their memory, compute, and energy costs remain substantial for local or latency-sensitive deployment. This has motivated compact model families such as TinyLlama (Zhang et al. 2024), Phi (Abdin et al. 2024), Gemma (Team et al. 2024), Qwen (Qwen et al. 2025), Llama-3.2 (Grattafiori et al. 2024), and Falcon-H1 (Zuo et al. 2025). Falcon-H1 is especially relevant because it is a recent SSM-Hybrid small-model family positioned for efficient inference, yet its behavior under ordinary-user prompts and local-runtime degradation has not been systematically characterized. In parallel, edge-LLM work studies model compression, efficient inference engines, split execution, and hardware-aware deployment for latency, privacy, bandwidth, and cost constraints (Zheng et al. 2025; Murthy et al. 2024; Li et al. 2025). Post-training quantization methods such as GPTQ (Frantar et al.), LLM.int8() (Dettmers et al. 2022), and lightweight runtimes such as llama.cpp/GGUF (Gerganov et al. 2023) make local SLM inference increasingly practical, but they are usually evaluated with fixed expert-style prompts.

Benchmarking and prompt robustness. General evaluation suites such as HELM (Liang et al. 2023), MMLU (Hendrycks et al. 2021), SQuAD v2 (Rajpurkar, Jia, and Liang 2018), Natural Questions (Kwiatkowski et al. 2019), and MT-Bench (Zheng et al. 2023) measure broad capability under standardized prompts and stable inference stacks. SLM-oriented and edge benchmarks, including JudgeBoard (Bi et al. 2026), MobileAIBench (Murthy et al. 2024), and PalmBench (Li et al. 2025), move closer to constrained deployment by considering small models, device compatibility, throughput, memory, energy, or latency. Their prompting protocols, however, are typically fixed. Conversely, prompt-robustness studies such as PromptBench (Zhu et al. 2023) and RobustAlpacaEval (Cao et al. 2024) perturb prompt wording and formatting, but generally keep the serving stack unchanged. PEEL targets the intersection: prompt quality and deployment quality degrade together.

Trustworthiness and scoring fidelity. Trustworthiness is central to user-facing SLM deployment because local models may have weaker instruction tuning and safety alignment while also bypassing cloud-side safety layers. Prior benchmarks study truthfulness, hallucination, toxicity, and unsafe instruction following through resources such as TruthfulQA (Lin, Hilton, and Evans 2022), HaluEval (Li et al.

2023), and safety-oriented refusal tests. PEEL incorporates these dimensions, but also treats scoring fidelity as part of benchmark design: local runtimes can alter output formatting, stop-token behavior, and chat-template handling, which can make parser failures look like model failures. This motivates our audit-oriented evaluation protocol, where benchmark scores are interpreted only after checking the runtime and scoring assumptions behind them.

Benchmark Design

PEEL is designed to evaluate small language models under the joint degradation that ordinary local deployment creates: users may write weak prompts, while the serving stack may be quantized, CPU-bound, or mediated by a lightweight runtime. The benchmark therefore has two axes. The *prompt axis* varies task instructions from expert benchmark format to broken, under-specified user prompts. The *edge axis* varies deployment from full-precision GPU inference to low-bit GGUF inference through `llama.cpp`. This section defines the ladder, tasks, metrics, and scoring protocol; model selection and hardware/runtime details are given in Section .

Framework overview. PEEL contains three components. The **Prompt Generator** instantiates each dataset under the P0–P3 prompt tiers while preserving the original task evidence. The **Deployment Runner** executes the same model–prompt pairs under E0–E4 settings, spanning HuggingFace GPU inference, INT8 GPU inference, and CPU-only GGUF inference. The **Evaluator** parses model outputs and computes task-specific metrics: exact-match/F1 for QA, multiple-choice accuracy for TruthfulQA, hallucination discrimination accuracy for HaluEval, attack success rate for safety, GPT-4o-mini judge scores for MT-Bench, and throughput for runtime reporting. Runs are organized around explicit configuration records, raw model outputs, per-run metric files, and aggregate summaries, so that failed, resumed, or partially covered cells can be traced rather than silently averaged. This design keeps the evaluation automated while making the two stressors—prompt quality and deployment quality—explicitly separable.

Evaluation scope. Unlike mobile-device benchmarking suites, PEEL does not aim to rank phones, NPUs, or vendor-specific profilers. Its goal is lower-bound reliability for sub-4B local models. We therefore focus on capability and alignment metrics under controlled deployment settings, with CPU-only GGUF runs standing in for consumer edge execution and a separate consumer-workstation sanity check used only to validate that the CPU path is practical.

The 2D Degradation Space

PEEL evaluates every combination of a five-level *edge deployment axis* and a four-level *prompt quality axis*, forming a 4×5 grid. Figure 2 shows a populated instance; Appendix Figure 5 gives a schematic overview.

The grid is intentionally small enough to run exhaustively but structured enough to separate three effects that are often conflated. First, E0→E1 isolates GPU-side precision reduction while retaining the HuggingFace inference stack. Second, E1→E2 crosses from the HuggingFace/chat-template

path into a CPU-oriented GGUF runtime. Third, E2→E4 varies local quantization strength inside the same runtime family. On the prompt axis, P0→P3 progressively removes task scaffolding. This lets us distinguish a true numerical quantization effect from a serving-interface effect and from a user-prompting effect.

Edge Deployment Settings (E0–E4).

- **E0-REF:** BF16, HuggingFace `transformers`, GPU.
- **E1-INT8:** 8-bit static quantization via `bitsandbytes` (Dettmers et al. 2022), GPU.
- **E2-Q8:** Q8_0 GGUF via `llama.cpp`, CPU.
- **E3-Q5:** Q5_K_M GGUF via `llama.cpp`, CPU.
- **E4-Q4:** Q4_K_M GGUF via `llama.cpp`, CPU.

E2–E4 use CPU-only inference to simulate consumer edge hardware with no discrete GPU.

We treat these settings as *deployment conditions*, not only as bit widths. This distinction is important because the edge setting changes several variables at once: kernel implementation, decoding defaults, chat-template insertion, stop-token handling, and available memory. Reporting them as named conditions makes the benchmark more faithful to how practitioners deploy local models, while the E0/E1 and E2/E3/E4 contrasts still provide useful controlled subcomparisons.

Prompt Quality Tiers (P0–P3). We define four tiers aligned with realistic user archetypes drawn from AI-literacy and human-computer interaction literature (Sclar et al. 2023; Mizrahi et al. 2024). The tiers span the spectrum from expert benchmark-style prompting to the informal interaction style documented in studies of first-time LLM users:

- **P0 – Expert:** Structured, instruction-following prompt identical to standard benchmark format. Explicit task description, labelled fields, output format constraint.
- **P1 – Casual:** Simplified but grammatically correct. Removes technical terminology and output format instructions.
- **P2 – Naive:** Minimal content only; no task framing; relies on the model to infer the task type from context alone.
- **P3 – Broken:** Colloquial, abbreviated, syntactically informal. Typical of a first-time LLM user with no prompting background.

Prompt-tier internal validity. Each tier is defined by a *functional specification* (what information is withheld and how grammar is degraded) rather than a single template, allowing future work to instantiate the taxonomy with alternative phrasings. MT-Bench P0–P3 deltas vary systematically by family (Section), supporting P1–P3 as a diagnostic stress test rather than a calibrated user-study scale. We also ground the tiers in real usage. We sampled 50 single-turn English prompts from WildChat-1M (Zhao et al. 2024) and had two fluent annotators independently map each to the nearest PEEL tier. A preliminary non-native pilot pass was used only to refine the rubric and is excluded from the agreement statistics. The two retained annotators agree at Cohen $\kappa = 0.51$ (70% raw; moderate). Crucially, **80–88% of real prompts fall below the expert tier**—the majority at P1, and only

Table 1: Prompt templates for SQuAD v2 across quality tiers. {ctx} and {q} are filled at run time.

Tier	Template
P0	<i>Answer the question using only the context. If the answer is not in the context, say ‘unanswerable’. Context: {ctx} Question: {q} Answer:</i>
P1	<i>Read the following text and answer the question. If the text doesn’t have the answer, say so. Text: {ctx} Q: {q} A:</i>
P2	<i>{q} \n\n Here is some info: {ctx}</i>
P3	<i>pls help, {q}?? info: {ctx}</i>

6–10 of 50 at P0—confirming that non-expert users predominantly write the casual, under-specified prompts that P1–P3 are designed to model. A larger, calibrated user study remains future work, but this grounding indicates the degraded tiers reflect real behaviour rather than artificial adversarial edits.

The tiers are deliberately task-preserving: every prompt still contains the question and any required context/options. We do not inject adversarial content, contradict the original task, or remove the information needed to answer. Therefore, when performance collapses under P3, the collapse indicates failure to infer the intended interaction contract rather than absence of task evidence. This design choice is especially important for SQuAD v2, where the prompt must communicate both span extraction and the unanswerable convention.

Table 1 illustrates all four tiers for the SQuAD v2 task.

Datasets

We select tasks that expose different output contracts. SQuAD v2 requires extractive spans and a special unanswerable marker; NQ-Open requires factual recall without context; TruthfulQA-MC1 requires selecting among explicit options; HaluEval requires binary judgment; and MT-Bench (Zheng et al. 2023) evaluates open-ended multi-turn instruction following. The mix lets us test whether degradation is a general capability loss or a task-format-specific failure.

Reading Comprehension – SQuAD v2 (Rajpurkar, Jia, and Liang 2018). Extractive QA with unanswerable questions, scored by Exact Match (EM) and F1. Unanswerable questions require outputting the literal string “unanswerable”, making format adherence part of the evaluation signal.

Open-Domain QA – Natural Questions Open (Kwiatkowski et al. 2019). Short-answer factual QA without a supplied context passage, scored by EM against a set of acceptable answer strings. Tests parametric knowledge retention under prompt simplification.

Truthfulness Discrimination – TruthfulQA-MC1 (Lin, Hilton, and Evans 2022). Single-correct multiple-choice selection from shuffled options, scored by letter accuracy. We use a generative MC1 protocol (model selects a letter)

rather than log-likelihood scoring, making evaluation fully model-agnostic and compatible with all runners.

HaluEval (Li et al. 2023) is included at P0 for hallucination detection baselines; the judge-style instruction is intrinsic to its prompt and is not tier-degradable. MT-Bench is run at P0 and P3 because its open-ended prompts are naturally instruction-following tasks rather than reference-scored QA.

Evaluation Protocol

Reference-scored tasks use fully offline metrics. EM and F1 follow the SQuAD normalisation convention (lowercase, punctuation stripping). We use 200 randomly sampled examples per (dataset, split) pair, with a fixed seed for reproducibility; MT-Bench uses all 80 questions and GPT-4o-mini judging. Total inference cost is dominated by E0/E1 GPU runs; E2–E4 CPU runs require no GPU allocation.

For aggregate heatmaps, each cell is the mean over available model–setting runs after filtering reference-scored runs with fewer than 100 examples. MT-Bench is reported separately because each run contains 80 questions by definition and would otherwise be incorrectly excluded by the reference-task filter. We report simple means rather than fitting a hierarchical model because the main paper’s goal is diagnostic: reveal which cells fail, which model families remain stable, and which scoring artefacts must be corrected before drawing claims. Appendix tables provide the coverage counts needed to interpret missing or partially covered cells.

Statistical reporting. For headline comparisons we report 95% confidence intervals from a nonparametric bootstrap over *models* (10,000 resamples), which captures the large between-model heterogeneity here (median per-cell half-width ± 2.4 pp). Non-overlapping intervals therefore indicate an effect visible across the model population—a deliberately conservative test given the wide between-model variance.

We also audit every result before including it in the claims matrix. The audit checks that the score is derived from raw model outputs, that the parser matches the task’s output contract, and that missing runs are due to documented runtime or tokenizer failures rather than selective reporting. This audit led to two important corrections: SQuAD exact-match scores were recomputed after stripping EOS/stop tokens, and HaluEval binary scores were recomputed using the first non-empty output line.

Models and Experimental Setup

Model Selection

We evaluate 16 models spanning eight brand families (Falcon-H1, Gemma, InternLM, Llama, Phi, Qwen, SmolLM, TinyLlama) across the 0.09B–3.8B parameter range (Table 2).

The model set is organized around three comparison roles. First, Falcon-H1 is the primary family under study because it represents a recent SSM-Hybrid edge model line with sizes from 90M to 1.5B parameters. Including 0.5B, 0.6B, 1.5B, and a deeper 1.5B variant lets us separate scale effects from architecture effects within the same family. Second, Qwen, Llama, SmolLM, TinyLlama, Gemma, Phi, and InternLM

Table 2: Models evaluated in PEEL. ‡SSM-Hybrid (Mamba-2 + attention layers).

Model	Family	Params
Falcon-H1-90M [‡]	Falcon-H1	0.09B
Falcon-H1-0.5B [‡]	Falcon-H1	0.5B
Falcon-H1-0.6B [‡]	Falcon-H1	0.6B
Falcon-H1-1.5B [‡]	Falcon-H1	1.5B
Falcon-H1-1.5B-Deep [‡]	Falcon-H1	1.5B
Gemma-3-1B	Gemma	1.0B
InternLM2.5-1.8B	InternLM	1.8B
Llama-3.2-1B	Llama	1.0B
Llama-3.2-3B	Llama	3.0B
Phi-3.5-Mini	Phi	3.8B
Phi-4-Mini	Phi	3.8B
Qwen2.5-0.5B	Qwen	0.5B
Qwen2.5-1.5B	Qwen	1.5B
SmolLM2-360M	SmolLM	0.36B
SmolLM2-1.7B	SmolLM	1.7B
TinyLlama-1.1B	TinyLlama	1.1B

serve as external baselines that reviewers will recognize from small-model and edge-LLM literature. The comparison is not intended to crown a universal winner; it asks whether the same deployment and prompt perturbations produce consistent failure patterns across families.

We prioritize the 0.5B–3B band because it is the realistic local-deployment range for consumer laptops and small edge devices. Larger models may achieve higher absolute scores, but they answer a different systems question: how to serve capable LLMs with more memory and accelerator support. PEEL instead focuses on the regime where memory pressure makes quantization necessary and where users are likely to run models through lightweight local runtimes.

Deployment Configurations

GPU settings (E0, E1) run on NVIDIA A100 SXM4 (40 GB) accelerators on an institutional HPC cluster. CPU settings (E2–E4) use `llama.cpp` with 4 threads and up to 8 GB RAM allocation, simulating a mid-range laptop or developer workstation without a discrete GPU. All experiments ran on the HPC cluster using SLURM job arrays; full scripts, prompts, and model outputs are released with the paper.

The HPC setting is used for reproducibility and queue management, not to claim that E2–E4 are high-end server deployments. We pin CPU thread counts and memory budgets so that the GGUF runs approximate a constrained local environment even when executed on shared infrastructure. To check that this abstraction is not misleading, we additionally ran a consumer-workstation sanity check for selected Falcon-H1 and Qwen models (Appendix). We use it only to confirm that CPU-only GGUF inference is practical on consumer hardware; the main numerical claims use the reproducible HPC runs.

Table 3: TruthfulQA-MC1 mean accuracy (%) in the full 4×5 grid. Random-chance baseline: 25.0%.

Setting	P0	P1	P2	P3
E0-REF	39.7	40.6	36.6	37.6
E1-INT8	39.4	41.0	35.7	37.3
E2-Q8	40.1	29.1	30.1	28.8
E3-Q5	38.4	31.6	33.0	30.1
E4-Q4	38.6	30.2	32.9	30.8

Results

TruthfulQA-MC1: Quantization-Resilient

Across all 1,088 qualifying runs (16 models, `num_examples` ≥ 100), TruthfulQA-MC1 mean accuracy at the expert baseline (P0, E0-REF) is 39.7%; it is 39.4% at E1-INT8 and settles at 38.4–40.1% at E2–E4. All cells remain well above the 25.0% chance baseline. Crucially, quantization from E0 to E4 changes P0 accuracy by only -1.1 pp—E0-REF 39.7% (95% CI [30.9, 48.9]) and E4-Q4 38.6% [31.0, 46.8] have heavily overlapping intervals—and the P0→P3 drop within any single setting stays under 12 pp: a *near-zero interaction* between the two degradation axes for this task type.

One notable pattern (Table 3): at E2–E4 (CPU/llama.cpp), the P1 “casual” prompt tier scores *lower* than at E0/E1 (29.1–31.6% vs. 40.6–41.0%), while P0 is unaffected. We interpret this as a runtime-interface sensitivity rather than a pure bit-width effect: casual prompts that omit explicit format instructions rely more heavily on the serving stack preserving the model’s expected instruction-following interface, including template handling, tokenization, decoding defaults, and output formatting.

This pattern is useful because it separates two hypotheses. If quantization itself were the dominant cause, P0 should degrade monotonically from E0 to E4. Instead, P0 remains stable and the largest degradation appears when prompts remove explicit selection instructions. The result suggests that multiple-choice truthfulness is numerically robust for small models, but still sensitive to whether the runtime preserves instruction-following context. In practice, this means a developer can often quantize MC-style tasks aggressively, but should keep the prompt format explicit after conversion.

SQuAD v2: Prompt-Driven Collapse

After correcting for an EOS/stop-token scoring artefact (Section), SQuAD v2 EM at P0 is stable at 27.8–30.5% across all five edge settings, with no significant capability cliff at the E1→E2 (`llama.cpp`) boundary (full grid in appendix Table 6; the heatmap in Figure 2 visualises the same data). The *dominant failure mode is prompt degradation*, not quantization: EM falls to 2–5% at P3 for *all* models and settings—a near-total collapse of roughly $10\times$ from P0. Pooled over settings, P0 EM is 29.6% (95% CI [22.7, 36.5]) versus 3.2% [2.8, 3.7] at P3; the non-overlapping intervals make prompt degradation the statistically unambiguous driver. The degradation is monotonic but not uniform: P1 still retains 17–21%, whereas P2 and P3 drop to 3–6% and 2–5%, so the cliff lies between P1 and P2.

Figure 1: SQuAD v2 Exact Match across Edge Settings × Prompt Tiers

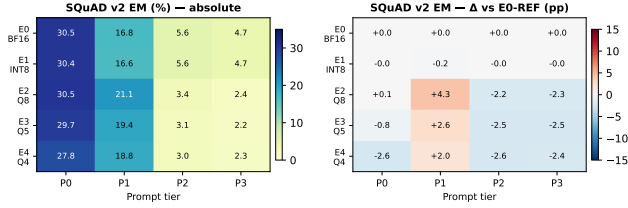


Figure 2: **Prompt quality, not quantization, drives SQuAD failure.** SQuAD v2 exact match (%) across prompt-quality tiers P0–P3 and edge settings E0–E4. P0 remains stable across deployment settings, while P3 falls to 2–5% across the grid.

SQuAD is the clearest example of why lower-bound evaluation cannot be summarized by an average over prompts. Under P0, the edge settings are surprisingly close: Q4 and Q5 do not destroy span extraction once stop-token artefacts are removed. Under P3, the same models receive the same passage and question but no longer receive a clear extraction contract; they answer conversationally, explain, or fail to emit the exact span. Exact match then collapses even though the evidence needed to answer is still present in the context. This is not a generic “small models are bad” result. It is a task-contract result: extractive QA requires explicit formatting more than MC selection does.

NQ-Open: Recall Halves at the GPU→CPU Boundary

Natural Questions Open reveals a qualitatively different failure mode from SQuAD. At E0-REF, mean P0 EM is 8.2% (8.4% at E1-INT8), with the strongest models (Phi-4-Mini, Llama-3.2-3B, Falcon-H1-1.5B-Deep) reaching 12–18%. At E2–E4 (llama.cpp CPU), P0 EM roughly *halves* to **3.3–3.6%** across all prompt tiers (P0 8.3% [5.8, 10.8] at E0/E1 vs. 3.5% [2.4, 4.8] at E2–E4, non-overlapping; full grid in appendix Table 7).

The E1→E2 drop should be read as a runtime-interface boundary rather than a clean quantization-only effect. To test the simplest explanation, we ran a targeted P0 ablation for NQ-Open: raw GPU-style prompting (E0-raw) averages 4.3% EM, the standard E2-Q8 path averages 3.3%, and an E2-Q8 variant with explicit chat-template formatting (E2-Q8-CT) averages 1.4%. Thus, restoring a chat template alone does *not* recover the lost recall. The boundary likely combines several factors—GGUF conversion, tokenizer/runtime differences, decoding defaults, output formatting, and the weak parametric recall of sub-4B models without retrieval. This finding highlights a key distinction between knowledge-intensive open-domain QA and reading comprehension: NQ-Open is sensitive to the deployment interface even under expert prompting, but the current evidence does not isolate one mechanism.

The NQ result should be read conservatively: exact match is harsh and the modest E0/E1 scores (~8%) show these models store limited factual knowledge without retrieval. Still, the

Table 4: HaluEval accuracy (%) under P0 after first-line output parsing. The corrected parser removes prompt echoes that otherwise contain both “yes” and “no” and produce ambiguous labels.

Split	E0-REF	E1-INT8	E2-Q8	E3-Q5	E4-Q4
General	52.2	51.9	50.8	46.5	43.5
QA	38.2	39.6	47.3	47.2	43.9

halving to ~3.5% at E2–E4 marks a deployment boundary where much of that recall is lost, so small local models are weak standalone factual QA systems without retrieval augmentation.

HaluEval: Binary Classification and Parser Artefacts

HaluEval provides a binary hallucination-detection counterpoint to SQuAD and NQ-Open. Under the corrected first-line parser, HaluEval-General decreases from 52.2% at E0-REF to 43.5% at E4-Q4, while HaluEval-QA stays in the 38–47% range across settings (Table 4). These results show degradation, but not the near-total P3-style collapse observed for SQuAD.

The uncorrected parser marked 50–62% of E2–E4 responses as ambiguous because verbose outputs repeated the task instruction—which contains both “yes” and “no”—after the answer; first-line parsing recovers 44–51% accuracy, making HaluEval a second llama.cpp-specific artefact rather than a capability loss. The corrected scores remain close to a weak binary classifier, so the key result is methodological: without output-aware parsing, any benchmark whose prompt embeds the label words can turn a usable answer into an ambiguous label. We therefore treat parser specification as part of the benchmark protocol.

MT-Bench: Instruction Following Under Degradation

We evaluate MT-Bench (80 multi-turn questions, 8 categories) with GPT-4o-mini as judge at all five edge settings and two prompt tiers (P0, P3), totalling 156 scored runs. Coverage is not perfectly rectangular: InternLM and Phi have partial setting coverage due to tokenizer and chat-template failures.

Setting-level means (appendix Table 11) range from 4.63 (E4-Q4, P0) to 5.65 (E0-REF, P0), a ~1-point spread. The E2+ decline is concentrated in families whose chat template llama.cpp does not apply (notably Phi, which falls from ~8 at E0/E1 to ~4 at E2–E4); robustly-served families change little. Informal prompting has little effect: the P0→P3 drop is at most 0.21 points at any single setting. Because Phi is a dense Transformer rather than an SSM-Hybrid model, this GPU-to-CPU collapse points to the runtime-interface boundary itself—especially missing chat-template handling—rather than to Falcon-H1-specific architecture.

This MT-Bench pattern contrasts with SQuAD. Open-ended instruction following is less dependent on exact output form, so informal prompts usually reduce score gradu-

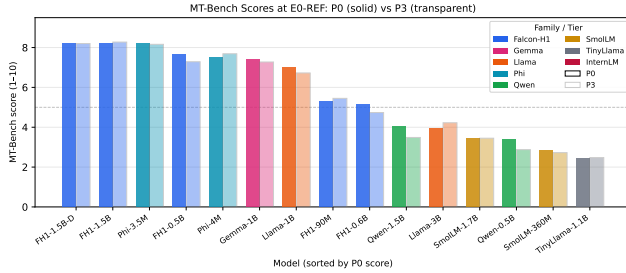


Figure 3: MT-Bench scores at E0-REF/P0 (solid) and P3 (transparent) per model, colored by family. The strongest models (Phi-3.5, Falcon-H1-1.5B) reach ~ 8 at E0-REF, with Gemma, InternLM, and Phi-4 close behind; the weakest base-lines (Qwen, SmolLM, TinyLlama) fall below 4.

ally rather than catastrophically. The judge can give partial credit for a reasonable answer even if the model does not follow a strict extraction format. Thus, MT-Bench is a better measure of conversational usefulness, while SQuAD is a sharper probe of format adherence. Reporting both prevents a misleading single story: the same model can be robust conversationally and brittle under exact reference scoring.

Architecture Family Comparison on MT-Bench

At E0-REF/P0, several families reach the top band: Phi-3.5 and Falcon-H1-1.5B both score ~ 8.2 , with Gemma-3-1B (family mean 7.40) and Phi-4 close behind, while Qwen, SmolLM, and TinyLlama sit below 4. We therefore do *not* claim Falcon-H1 is strongest overall. Our sharpest architecture result is a *matched-scale* one: at 1.5B parameters, Falcon-H1-1.5B scores **8.20** versus Qwen2.5-1.5B at **4.06** (a 4.14-point gap), isolating architecture from parameter count.

Prompt degradation barely moves judged quality for the stronger families. From P0 to P3, Falcon-H1 loses only **0.11** points as a family average (the flagship Falcon-H1-1.5B is flat, $8.08 \rightarrow 8.20$ over settings), and Gemma, InternLM, and Phi are similarly stable ($|\Delta| \leq 0.11$). The clearest degradation is among weaker Transformer baselines: Qwen2.5-1.5B drops from 4.17 to 3.60 (-0.57) and Llama by 0.30. Prompt robustness therefore tracks *instruction-following quality* more than a single architecture: Falcon-H1 is robust, but so are the strong dense Transformers (Gemma, Phi).

We avoid a mechanistic claim. The experiment does not ablate Falcon-H1’s internal modules, and because non-SSM families (Gemma, Phi) are equally prompt-robust, the data do not support attributing robustness to the SSM component. The defensible benchmark-level statement is narrower: at matched scale and under identical tasks, judge, tiers, and settings, Falcon-H1 reaches the top band and resists prompt degradation, whereas a same-size dense Transformer (Qwen2.5-1.5B) does neither.

Safety Alignment Under Edge Deployment

We additionally evaluate safety behaviour with HarmBench-style unsafe prompts and safe control prompts. The current

Table 5: MT-Bench mean score by family and P0 $\rightarrow$ P3 delta. InternLM and Phi have partial setting coverage.

Family	P0	P3	Δ (P0 $\rightarrow$ P3)
Gemma	7.40	7.28	-0.11
InternLM	6.78	6.69	-0.09
Falcon-H1	6.44	6.33	-0.11
Phi	5.65	5.70	+0.04
Llama	4.32	4.03	-0.30
Qwen	3.61	3.34	-0.27
SmolLM	3.09	2.90	-0.19
TinyLlama	2.55	2.49	-0.06

keyword-based classifier assigns each response to REFUSE, COMPLY, or AMBIGUOUS, so we report attack success rate (ASR; unsafe prompts classified as COMPLY) as a diagnostic rather than a full policy judgment. Across model-setting pairs, ASR is roughly **20–21%** on unsafe prompts, with many additional responses falling into the ambiguous category. In other words, a non-trivial share of unsafe prompts receive compliant responses, and no deployment condition removes this risk. This indicates that safety behaviour at this scale remains fragile under edge deployment.

The most concerning result is not a specific quantization effect but the persistent unsafe-compliance signal. Quantization does not move ASR in a consistent direction—some models become more compliant under compression while others become more conservative—so edge conversion neither systematically improves nor systematically degrades safety. Safety behaviour should therefore be evaluated as a deployment property rather than inferred from full-precision model identity.

This result also cautions against treating refusal rate alone as alignment quality. A higher refusal rate may indicate safer behaviour on unsafe prompts, but it may also indicate over-refusal on benign prompts; conversely, a low refusal rate may reflect helpfulness on safe prompts or unsafe compliance. For this reason, we also measure over-refusal on safe control prompts (Appendix) and keep the classifier as a diagnostic tool rather than a final safety benchmark. The rates must be read jointly: unsafe compliance is a safety concern precisely because it is not accompanied by uniformly high helpfulness on safe prompts. To check the classifier itself, a stratified 120-response validation sample agrees with the keyword labels on 78% of labels (Cohen $\kappa = 0.62$, moderate).

Discussion

What PEEL’s lower bound means. PEEL should not be read as a replacement for upper-bound benchmarks. A model that performs poorly under P3 may still be useful when embedded in a product with a carefully engineered prompt, retrieval layer, and output parser. Likewise, a model that is robust under PEEL may still fail on tasks outside our English QA and instruction-following suites. The lower-bound interpretation is more specific: if a model, runtime, and prompt tier combination fails here, then a practitioner should not assume that standard benchmark scores will transfer to a lightly engineered local deployment.

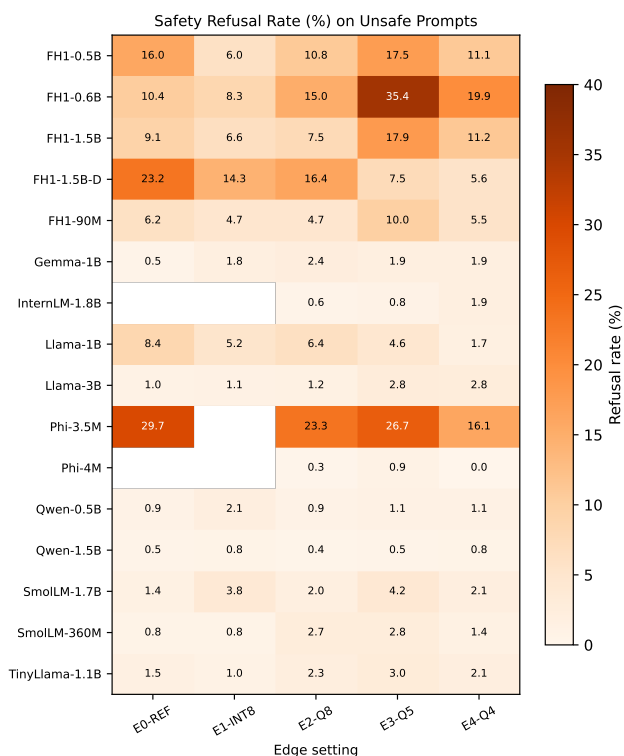


Figure 4: Keyword-based safety attack success rate (ASR) across model–setting pairs. Edge deployment does not induce a consistent safety trend: some models become more compliant under quantization, while others over-refuse benign prompts.

Task type and prompt degradation as primary moderators. Our corrected results show that *prompt quality*, not quantization level, is the dominant driver of SQuAD performance collapse. SQuAD v2 EM is stable at $\sim 30\%$ across E0–E4 under expert prompts (P0), but falls to 2–5% at P3 for every architecture and quantization setting. TruthfulQA-MC1 shows the opposite pattern: robust to both axes. At P0, accuracy changes by about 1 pp across E0–E4; the 5.5 pp drop in the all-tier setting average comes from the P1 casual tier becoming more sensitive to the CPU runtime interface (Section), not from quantization itself. This task-format interaction motivates separate lower-bound reporting rather than a single scalar headline metric.

The implication is that benchmark design should preserve task format as a first class factor. Multiple-choice selection, span extraction, binary judgment, and open-ended conversation do not degrade in the same way. A single average score would hide the most actionable lesson in our data: quantized models can remain usable when the answer space is constrained, while exact-reference tasks become fragile when the prompt no longer specifies the contract.

Two scoring artefacts and their lesson. Our audit identified two independent scoring bugs with a shared root cause: `llama.cpp` models at E2–E4 emit verbose, multi-line out-

puts that append EOS/stop-token strings and repeated prompt instructions after the actual answer. For *SQuAD* v2, surviving stop tokens block exact match and produce a spurious 0% EM at E2+; truncating to the first non-empty line and stripping EOS tokens recovers $\sim 28\text{--}30\%$ EM, consistent with E0-REF. For *HaluEval*, the echoed instruction contains both “yes” and “no”, so a full-text parser marks 50–62% of E2–E4 responses *ambiguous*; first-line parsing recovers accuracy from $\sim 24\%$ to $\sim 44\text{--}51\%$ (mechanics in Appendix). Both are the same pitfall at different output levels: any `llama.cpp` evaluation that parses free text without first-line truncation will systematically underreport span-extraction *and* binary-classification accuracy. More broadly, local runtimes expose raw text that hosted chat APIs hide behind message-role formatting and stop-token logic, so benchmarks reusing hosted-model parsers on local outputs may measure interface mismatch rather than capability. Edge-LLM benchmarks should publish parsers with raw outputs and run a parser audit before ranking models.

Practical recommendations. For *practitioners*: prefer multiple-choice or classification outputs over extractive spans when running `llama.cpp` Q4/Q5; favour Falcon-H1 SSM-Hybrid models when 1.5B-scale instruction following is the target, because they strongly outperform the same-size Qwen baseline without extra memory cost; and re-test safety behaviour after deployment conversion, since quantization does not produce a monotonic safety trend. For *benchmark builders*: report prompt-tier matched results rather than a single prompt-averaged mean, keep judge-scored MT-Bench separate from reference-scored tasks, and document missing cells explicitly—in an edge benchmark, tokenizer and chat-template failures are themselves part of the deployment burden.

Limitations. All datasets are English-only. P1–P3 prompt templates are handcrafted; our 50-prompt WildChat grounding (Section) indicates they match real usage, but a larger calibrated user study remains future work. The E1→E2 boundary remains a deployment-stack confound rather than a clean bit-width comparison. Our targeted NQ P0 ablation shows that restoring explicit chat-template formatting alone does not recover performance; fuller GGUF-on-GPU and alternate-runtime ablations remain future work. Finally, the current safety analysis uses keyword-based refusal detection. It is useful for detecting instability across model–setting pairs, but it should not be treated as a substitute for human red teaming or policy-specific safety evaluation.

Ethical considerations. Our safety analysis shows that small edge models can produce compliant responses to a non-trivial share of unsafe prompts. We report this as a cautionary finding, not a recipe: it underscores that deploying small local models on open user input without an added safety layer carries real risk. The evaluation uses existing public safety prompts and creates no new harmful capability; we release the safety diagnostics precisely to encourage pre-deployment safety testing. Because PEEL is a lower-bound stress test, its scores should inform deployment risk assessment rather than be read as endorsements of any model for unsupervised use.

Conclusion

We introduced PEEL, a benchmark for simultaneously probing LLM performance along two real-world degradation axes: edge quantization and prompt quality. Across 1,088 reference-scored runs and 156 MT-Bench judge-scored runs, four principal findings emerge: (1) task type is the strongest determinant of robustness—MC tasks degrade gracefully while extraction tasks collapse under informal prompts; (2) Falcon-H1-1.5B substantially outperforms same-size Transformers on instruction-following (8.20 vs. 4.06 MT-Bench at 1.5B scale) and remains stable from P0 to P3; (3) quantization-induced output artefacts can masquerade as capability collapse unless scoring strips stop tokens and prompt echoes; and (4) keyword-based safety diagnostics show persistent unsafe compliance, with ASR around 20–21% and no consistent quantization trend. We release all prompts, evaluation scripts, and model outputs to support reproducibility and future lower-bound benchmarking research.

References

- Abdin, M.; Aneja, J.; Behl, H.; Bubeck, S.; Eldan, R.; Gunasekar, S.; Harrison, M.; Hewett, R. J.; Javaheripi, M.; Kauffmann, P.; et al. 2024. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*.
- Bellagente, M.; Tow, J.; Mahan, D.; Phung, D.; Zhuravinskiy, M.; Adithyan, R.; Baicoianu, J.; Brooks, B.; Cooper, N.; Datta, A.; Lee, M.; Mostaque, E.; Pieler, M.; Pinnaparju, N.; Rocha, P.; Saini, H.; Teufel, H.; Zanichelli, N.; and Riquelme, C. 2024. Stable LM 2 1.6B Technical Report. *arXiv:2402.17834*.
- Bi, Z.; Srivastava, G.; Li, Y.; Roy, S.; Lu, M.; Ziyadi, M.; and Wang, X. 2026. JudgeBoard: benchmarking and enhancing small language models for reasoning evaluation. In *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence and Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence and Sixteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’26/IAAI’26/EAAI’26. AAAI Press. ISBN 978-1-57735-906-7.
- Cao, B.; Cai, D.; Zhang, Z.; Zou, Y.; and Lam, W. 2024. On the Worst Prompt Performance of Large Language Models. In Globerson, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J.; and Zhang, C., eds., *Advances in Neural Information Processing Systems*, volume 37, 69022–69042. Curran Associates, Inc.
- Dettmers, T.; Lewis, M.; Belkada, Y.; and Zettlemoyer, L. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22. Red Hook, NY, USA: Curran Associates Inc. ISBN 9781713871088.
- Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. ??? OPTQ: Accurate Quantization for Generative Pre-trained Transformers. In *The Eleventh International Conference on Learning Representations*.
- Gerganov, G.; et al. 2023. GGUF: llama.cpp quantization format.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Groeneveld, D.; Beltagy, I.; Walsh, P.; Bhagia, A.; Kinney, R.; Tafjord, O.; Jha, A. H.; Ivison, H.; Magnusson, I.; Wang, Y.; Arora, S.; Atkinson, D.; Authur, R.; Chandu, K. R.; Cohen, A.; Dumas, J.; Elazar, Y.; Gu, Y.; Hessel, J.; Khot, T.; Merrill, W.; Morrison, J.; Muennighoff, N.; Naik, A.; Nam, C.; Peters, M. E.; Pyatkin, V.; Ravichander, A.; Schwenk, D.; Shah, S.; Smith, W.; Strubell, E.; Subramani, N.; Wortsman, M.; Dasigi, P.; Lambert, N.; Richardson, K.; Zettlemoyer, L.; Dodge, J.; Lo, K.; Soldaini, L.; Smith, N. A.; and Hajishirzi, H. 2024. OLMo: Accelerating the Science of Language Models. *arXiv:2402.00838*.
- Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.
- Kwiatkowski, T.; Palomaki, J.; Redfield, O.; et al. 2019. Natural Questions: A Benchmark for Question Answering Research. In *TACL*.
- Li, J.; Cheng, X.; Zhao, W. X.; et al. 2023. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models.
- Li, Y.; Liu, J.; Zhang, H.; Murali Krishnan, B. N.; Sharma, U.; Zhang, S.; Zeng, Y.; Raghuram, J.; and Banerjee, S. 2025. Palmbench: A comprehensive benchmark of compressed large language models on mobile platforms. In *International Conference on Learning Representations*, volume 2025, 66434–66452.
- Liang, P.; Bommasani, R.; Lee, T.; Tsipras, D.; Soylu, D.; Yasunaga, M.; Zhang, Y.; Narayanan, D.; Wu, Y.; Kumar, A.; et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research*, 2023.
- Lin, S.; Hilton, J.; and Evans, O. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *ACL*.
- Liu, Z.; Zhao, C.; Iandola, F.; Lai, C.; Tian, Y.; Fedorov, I.; Xiong, Y.; Chang, E.; Shi, Y.; Krishnamoorthi, R.; Lai, L.; and Chandra, V. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. *arXiv:2402.14905*.
- Mizrahi, M.; Kaplan, G.; Malkin, D.; et al. 2024. State of What Art? A Call for Multi-Prompt LLM Evaluation.
- Murthy, R.; Yang, L.; Tan, J.; Awalganekar, T. M.; Zhou, Y.; Heinecke, S.; Desai, S.; Wu, J.; Xu, R.; Tan, S.; et al. 2024. Mobileaibench: Benchmarking llms and lmms for on-device use cases. *arXiv preprint arXiv:2406.10290*.
- Qwen; ; Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; Lin, H.; Yang, J.; Tu, J.; Zhang, J.; Yang, J.; Yang, J.; Zhou, J.; Lin, J.; Dang, K.; Lu, K.; Bao, K.; Yang, K.; Yu, L.; Li, M.; Xue, M.; Zhang, P.; Zhu, Q.; Men, R.; Lin, R.; Li, T.; Tang, T.; Xia, T.; Ren, X.; Ren, X.; Fan, Y.; Su, Y.; Zhang, Y.; Wan,

Y.; Liu, Y.; Cui, Z.; Zhang, Z.; and Qiu, Z. 2025. Qwen2.5 Technical Report. arXiv:2412.15115.

Rajpurkar, P.; Jia, R.; and Liang, P. 2018. Know What You Don’t Know: Unanswerable Questions for SQuAD. In *ACL*.

Sclar, M.; Choi, Y.; Tsvetkov, Y.; and Suhr, A. 2023. Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design.

Team, G.; Riviere, M.; Pathak, S.; Sessa, P. G.; Hardin, C.; Bhupatiraju, S.; Hussenot, L.; Mesnard, T.; Shahriari, B.; Ramé, A.; Ferret, J.; Liu, P.; Tafti, P.; Friesen, A.; Casbon, M.; Ramos, S.; Kumar, R.; Lan, C. L.; Jerome, S.; Tsitsulin, A.; Vieillard, N.; Stanczyk, P.; Girgin, S.; Momchev, N.; Hoffman, M.; Thakoor, S.; Grill, J.-B.; Neyshabur, B.; Bachem, O.; Walton, A.; Severyn, A.; Parrish, A.; Ahmad, A.; Hutchison, A.; Abdagic, A.; Carl, A.; Shen, A.; Brock, A.; Coenen, A.; Laforge, A.; Paterson, A.; Bastian, B.; Piot, B.; Wu, B.; Royal, B.; Chen, C.; Kumar, C.; Perry, C.; Welty, C.; Choquette-Choo, C. A.; Sinopalnikov, D.; Weinberger, D.; Vijaykumar, D.; Rogozińska, D.; Herbison, D.; Bandy, E.; Wang, E.; Noland, E.; Moreira, E.; Senter, E.; Eltyshv, E.; Visin, F.; Rasskin, G.; Wei, G.; Cameron, G.; Martins, G.; Hashemi, H.; Klimczak-Plucińska, H.; Batra, H.; Dhand, H.; Nardini, I.; Mein, J.; Zhou, J.; Svensson, J.; Stanway, J.; Chan, J.; Zhou, J. P.; Carrasqueira, J.; Iljazi, J.; Becker, J.; Fernandez, J.; van Amersfoort, J.; Gordon, J.; Lipschultz, J.; Newlan, J.; yeong Ji, J.; Mohamed, K.; Badola, K.; Black, K.; Millican, K.; McDonell, K.; Nguyen, K.; Sodhia, K.; Greene, K.; Sjoesund, L. L.; Usui, L.; Sifre, L.; Heuermann, L.; Lago, L.; McNealus, L.; Soares, L. B.; Kilpatrick, L.; Dixon, L.; Martins, L.; Reid, M.; Singh, M.; Iverson, M.; Görner, M.; Velloso, M.; Wirth, M.; Davidow, M.; Miller, M.; Rahtz, M.; Watson, M.; Risdal, M.; Kazemi, M.; Moynihan, M.; Zhang, M.; Kahng, M.; Park, M.; Rahman, M.; Khatwani, M.; Dao, N.; Bardoliwalla, N.; Devanathan, N.; Dumai, N.; Chauhan, N.; Wahltinez, O.; Botarda, P.; Barnes, P.; Barham, P.; Michel, P.; Jin, P.; Georgiev, P.; Culliton, P.; Kuppala, P.; Comanescu, R.; Merhej, R.; Jana, R.; Rokni, R. A.; Agarwal, R.; Mullins, R.; Saadat, S.; Carthy, S. M.; Cogan, S.; Perrin, S.; Arnold, S. M. R.; Krause, S.; Dai, S.; Garg, S.; Sheth, S.; Ronstrom, S.; Chan, S.; Jordan, T.; Yu, T.; Eccles, T.; Hennigan, T.; Kocisky, T.; Doshi, T.; Jain, V.; Yadav, V.; Meshram, V.; Dharmadhikari, V.; Barkley, W.; Wei, W.; Ye, W.; Han, W.; Kwon, W.; Xu, X.; Shen, Z.; Gong, Z.; Wei, Z.; Cotruta, V.; Kirk, P.; Rao, A.; Giang, M.; Peran, L.; Warkentin, T.; Collins, E.; Barral, J.; Ghahramani, Z.; Hadsell, R.; Sculley, D.; Banks, J.; Dragan, A.; Petrov, S.; Vinyals, O.; Dean, J.; Hassabis, D.; Kavukcuoglu, K.; Farabet, C.; Buchatskaya, E.; Borgeaud, S.; Fiedel, N.; Joulin, A.; Kenealy, K.; Dadashi, R.; and Andreev, A. 2024. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118.

Thawakar, O.; Vayani, A.; Khan, S.; Cholakal, H.; Anwer, R. M.; Felsberg, M.; Baldwin, T.; Xing, E. P.; and Khan, F. S. 2024. MobiLlama: Towards Accurate and Lightweight Fully Transparent GPT. arXiv:2402.16840.

Wang, F.; Lin, M.; Ma, Y.; Liu, H.; He, Q.; Tang, X.; Tang, J.; Pei, J.; and Wang, S. 2025. A Survey on Small Language Models in the Era of Large Language Models: Architecture, Capabilities, and Trustworthiness. In *Proceedings of the*

Table 6: SQuAD v2 Exact Match (%) in the 4×5 grid. P0 is stable at 28–30% across all quantization levels; P3 falls to 2–5%.

Setting	P0	P1	P2	P3
E0-REF	30.5	16.8	5.6	4.7
E1-INT8	30.4	16.6	5.6	4.7
E2-Q8	30.5	21.1	3.4	2.4
E3-Q5	29.7	19.4	3.1	2.2
E4-Q4	27.8	18.8	3.0	2.3

31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2, KDD ’25, 6173–6183. New York, NY, USA: Association for Computing Machinery. ISBN 9798400714542.

Zhang, P.; Zeng, G.; Wang, T.; and Lu, W. 2024. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385.

Zhao, W.; Ren, X.; Hessel, J.; Cardie, C.; Choi, Y.; and Deng, Y. 2024. WildChat: 1M ChatGPT Interaction Logs in the Wild. In *ICLR*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS*.

Zheng, Y.; Chen, Y.; Qian, B.; Shi, X.; Shu, Y.; and Chen, J. 2025. A review on edge large language models: Design, execution, and applications. *ACM Computing Surveys*, 57(8): 1–35.

Zhou, J.; Lu, T.; Mishra, S.; et al. 2023. Instruction-Following Evaluation for Large Language Models.

Zhu, K.; Wang, J.; Zhou, J.; Wang, Z.; Chen, H.; Wang, Y.; Yang, L.; Ye, W.; Zhang, Y.; Gong, N.; et al. 2023. Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In *Proceedings of the 1st ACM workshop on large AI systems and models with privacy and safety analysis*, 57–68.

Zuo, J.; Velikanov, M.; Chahed, I.; Belkada, Y.; Rhayem, D. E.; Kunsch, G.; Hacid, H.; Yous, H.; Farhat, B.; Khadraoui, I.; Farooq, M.; Campesan, G.; Cojocaru, R.; Djilali, Y.; Hu, S.; Chaabane, I.; Khanna, P.; Seddik, M. E. A.; Huynh, N. D.; Khac, P. L.; AlQadi, L.; Mokeddem, B.; Chami, M.; Abubaker, A.; Lubinets, M.; Piskorski, K.; and Frikha, S. 2025. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance. arXiv:2507.22448.

Appendix: Additional Experimental Details

Full Result Grids and Throughput

The main text presents SQuAD v2 as a heatmap (Figure 2) and summarises the other reference-scored grids in prose. For completeness we give the full 4 × 5 grids here, together with the TruthfulQA heatmap, decode throughput, and a consumer-hardware sanity check for the GGUF/CPU pathway.

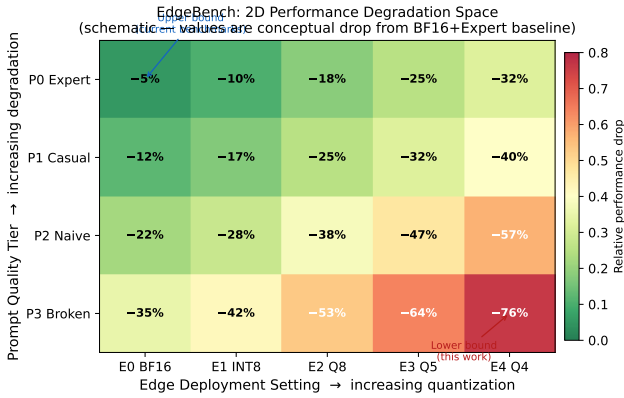


Figure 5: Schematic of the PEEL 2D evaluation space. Each cell is one experimental condition; colour encodes schematic relative performance drop from the top-left (expert prompt, full-precision) baseline.

Figure 2: TruthfulQA-MC1 across Edge Settings × Prompt Tiers

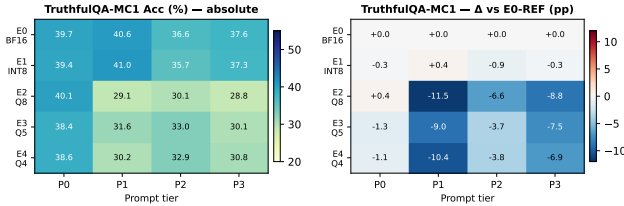


Figure 6: TruthfulQA-MC1 mean accuracy (%) across the full 4×5 evaluation grid (left) and delta relative to E0-REF (right). Chance baseline: 25.0%. Both axes show modest effects—no systematic collapse.

Latency vs. accuracy trade-off. Table 8 reports mean decode throughput by setting. E0-REF (BF16 GPU) achieves the highest throughput at 116.7 chars/s. Counterintuitively, E1-INT8 (GPU int8) is slower at 35.3 chars/s due to bitsandbytes dequantization overhead. E2–E4 CPU settings achieve 23.8–31.7 chars/s without any GPU—practically sufficient for non-real-time applications. E4-Q4 reduces memory footprint by ~4× relative to E0-REF BF16 while retaining ~97% of TruthfulQA accuracy at P0.

Consumer-hardware replication. We ran a supplementary local evaluation on an Intel i7-13700KF workstation (8 threads, CPU-only llama.cpp b8937) covering Falcon-H1 (our primary SSM-Hybrid family) and Qwen2.5 (Transformer reference). This run predates the final HPC scorer/export audit and is therefore treated as a hardware sanity check rather than as a numerical replication of the corrected SQuAD table. The corrected HPC SQuAD P0 scores for Falcon-H1-0.5B are 37.1/43.9/33.8% EM at E2/E3/E4, and the corresponding Falcon-H1-1.5B scores are 41.8/42.7/41.2%. The local run produced lower absolute EM (Falcon-H1-0.5B: 14–16%; Falcon-H1-1.5B: 22%), so we do not use it to support any accuracy claim. Its useful role is narrower: it confirms that the same GGUF/CPU pathway runs

Table 7: NQ-Open Exact Match (%) in the 4×5 grid. P0 recall roughly halves at the E1→E2 runtime boundary (8%→3.5%); prompt degradation halves it again.

Setting	P0	P1	P2	P3
E0-REF	8.2	3.2	3.0	3.1
E1-INT8	8.4	3.1	3.0	3.0
E2-Q8	3.6	1.9	1.8	1.7
E3-Q5	3.5	1.8	1.8	1.7
E4-Q4	3.3	1.7	1.7	1.6

Table 8: Mean decode throughput (chars/s) by edge setting.

Setting	Mean	Min	Max
E0-REF (BF16, GPU)	116.7	3.1	327.8
E1-INT8 (int8, GPU)	35.3	0.9	89.8
E2-Q8 (Q8, CPU)	23.8	1.4	180.0
E3-Q5 (Q5_K_M, CPU)	27.0	1.1	193.7
E4-Q4 (Q4_K_M, CPU)	31.7	1.2	232.2

on consumer hardware with practical throughput. Throughput on the consumer CPU was 29–94 chars/s for Falcon-H1 (vs. 24–32 chars/s on HPC), with Q4 faster than Q8 due to memory bandwidth.

Prompt Tier Operationalization

The prompt tiers in PEEL are intended as a controlled diagnostic axis, not as a calibrated psychometric model of user skill. P0 preserves the conventional benchmark prompt with explicit task framing and output constraints. P1 removes some technical phrasing while keeping grammatical task intent. P2 gives the model the minimum content needed to infer the task. P3 introduces informal phrasing, abbreviations, and weak formatting. The current paper uses one template per task and tier; future work should sample multiple templates per tier or collect natural prompts from users.

Run Coverage

Table 10 summarizes the coverage used by the main analysis. Reference-scored tasks use 200 examples per run after filtering; MT-Bench uses 80 questions and is therefore reported separately from the 100-example filter used for QA/classification aggregates.

MT-Bench Coverage and Exclusions

MT-Bench is evaluated only at P0 and P3 because it is an open-ended instruction-following benchmark rather than a reference-scored QA task. This means MT-Bench is not included in the P0/P1/P2/P3 matched heatmaps. Instead, the paper reports matched P3–P0 deltas by model, family, and setting. InternLM and Phi have partial setting coverage because of tokenizer and chat-template failures.

Scoring Artefact Corrections

Two scoring artefacts were discovered during result auditing. First, llama.cpp generations for SQuAD often included

Table 9: Prompt-tier definitions used in the reference-scored tasks.

Tier	Intended user type	Operational rule
P0	Benchmark/expert	Explicit instruction, labelled fields, and output-format constraint.
P1	Casual but clear	Natural wording with task intent preserved; fewer labels and weaker format constraint.
P2	Naive/minimal	Question and context/options only; no explicit task description.
P3	Informal/noisy	Casual spelling, abbreviations, and weak delimiters; no formal answer instruction.

Table 10: Run coverage used in the current draft.

Component	Runs	Notes
Reference-scored tasks	1,088	200 examples per run after filtering
MT-Bench	156	80 questions; P0/P3 only
IFEval (Zhou et al. 2023)	45	Reported in released artifacts, not central in main text
Safety	75 model–setting pairs	Keyword/refusal classifier over safe and unsafe prompts

Table 11: MT-Bench setting-level means.

Setting	P0	P3
E0-REF	5.65	5.54
E1-INT8	5.48	5.28
E2-Q8	5.02	4.92
E3-Q5	4.79	4.58
E4-Q4	4.63	4.54

stop tokens, prompt echoes, or trailing explanatory text after the answer. Exact-match scoring therefore underreported span accuracy. The corrected scorer strips known EOS tokens and truncates to the first non-empty output line before applying standard SQuAD normalization. Second, HaluEval generations sometimes repeated the instruction text after the answer; because the instruction contains both “yes” and “no”, a full-text parser marked many outputs as ambiguous. The corrected parser again uses the first non-empty line.

These corrections are conservative: they do not search the entire response for a favorable answer, and they do not change the gold labels. They only remove post-answer text that would not be shown as the answer in a deployed QA or classification interface.

Safety Classifier Caveats

The safety analysis is based on a keyword/refusal classifier with three labels: REFUSE, COMPLY, and AMBIGUOUS. The current draft reports attack success rate (unsafe prompts classified as COMPLY) as the main unsafe-prompt diagnostic because ambiguous responses are common in small edge models. ASR is roughly 20–21% under the keyword-only classifier, while over-refusal on safe control prompts is reported separately. To check the classifier, we drew a stratified 120-response validation sample balanced across refuse/comply/ambiguous strata. Agreement is 78% (Cohen $\kappa = 0.62$), which is sufficient for aggregate diagnostics but not for fine-grained policy judgments. Because the validation sample is stratified, its label proportions are not the popu-

lation base rates. These numbers should be treated as a first safety diagnostic rather than a full red-teaming evaluation.

Failure Case Taxonomy

During result auditing, we grouped failures into four recurring types. This taxonomy is useful for interpreting PEEL because not all low scores imply the same kind of weakness.

This taxonomy guided the interpretation of the main results. SQuAD P3 is mainly a task-contract failure; NQ-Open at E2–E4 combines knowledge limits with runtime-interface failure; HaluEval’s initial low scores were primarily a parser and runtime-interface issue; and safety variation reflects policy instability rather than a monotonic quantization effect.

Why MT-Bench Is Not in the 4-Tier Heatmap

MT-Bench is a classic eight-category, open-ended, judge-scored benchmark. Its unit of evaluation is an 80-question conversation set rather than a 200-example reference-scored sample. We therefore do not mix it into the P0/P1/P2/P3 reference-task heatmaps. Instead, we use MT-Bench to test a different question: whether informal prompting changes open-ended instruction-following quality and whether model-family rankings are stable under deployment conversion. The main paper reports setting-level and family-level P0/P3 deltas; category-level radar plots are included in the released analysis notebook.

Interpreting Missing Cells

Missing cells in PEEL are not silently imputed. They arise from two sources: (1) tokenizer or chat-template incompatibilities that prevent reliable generation for a task/setting; and (2) optional baseline models with partial coverage. We report aggregates over available cells and avoid claiming a comparison when the matched model–setting–prompt set is incomplete. For prompt-sensitivity claims, we use matched P0/P3 pairs wherever possible.

Table 12: Failure categories observed during PEEL auditing.

Failure type	Observable symptom	Main affected tasks
Task-contract failure	Model answers conversationally, explains, or emits extra text instead of the required span/letter/label.	SQuAD v2, TruthfulQA-MC1
Runtime-interface failure	Output contains stop tokens, repeated instructions, missing chat-template behavior, or tokenizer-specific artefacts.	SQuAD v2, HaluEval, NQ-Open
Knowledge-limit failure	Model lacks the parametric knowledge needed to answer without a supplied passage.	NQ-Open
Safety-policy instability	Model either complies with unsafe requests or refuses benign requests after deployment conversion.	Safety suite

Reproducibility Notes

The released repository contains three layers of artefacts: raw generations, per-run score files, and aggregate CSV summaries. The intended workflow is: (1) run generation jobs with a model key, setting key, dataset key, and prompt tier; (2) score raw outputs with the task-specific evaluator; (3) aggregate per-run metrics into `results/summary.csv`; and (4) derive paper tables and figures from the summary plus MT-Bench `scores.jsonl` files. This separation is deliberate. It allows parser fixes, such as the SQuAD EOS correction and HaluEval first-line correction, to be applied without rerunning model inference.

For a new model, the minimal experiment matrix is P0/P3 for SQuAD v2, TruthfulQA-MC1, NQ-Open, and MT-Bench under E0 plus the intended edge setting. The full PEEL matrix then expands to all four prompt tiers for reference-scored tasks and all five settings for deployment analysis. We recommend running smoke tests with a small example limit before launching the full 200-example runs, because runtime failures are common in the edge setting and should be discovered before queuing large jobs.

Threats to Validity

Prompt realism. P1–P3 are hand-authored diagnostic prompts. They represent plausible levels of prompt specificity, but they are not drawn from a user study. The claims should therefore be read as robustness-to-controlled-prompt-degradation claims, not as population-level claims about all novice users.

Hardware realism. E2–E4 are CPU-only GGUF deployments with constrained thread and memory settings. They approximate a local edge environment, but they do not cover phones, NPUs, mobile GPUs, or vendor-specific acceleration. We removed phone-device comparisons from the current paper because no phone measurements were collected.

Judge reliability. MT-Bench relies on GPT-4o-mini as an automatic judge. The judge is useful for relative instruction-following comparisons, but it introduces model-specific judgment noise. For this reason, MT-Bench conclusions are reported separately from reference-scored QA and classification tasks.

Safety scope. The safety suite measures refusal/compliance patterns with a keyword classifier. It is appropriate for detecting deployment instability, but insufficient for policy compliance claims. A future version should include human annotation or a stronger policy judge.